



**EAST WEST INSTITUTE OF TECHNOLOGY**  
(an Autonomous Institution under VTU, Belagavi)  
#63, off Magadi Road, Vishwaneedam Post, Bengaluru-560091.



# **DEPARTMENT OF MCA**

**LAB MANUAL FOR**

**II SEMESTER**

**ACADEMIC YEAR 2024-2025**

**JAVA LABORATORY**

**(PMCAL27)**

**COMPILED BY**  
**Prof. Sheethal P P, (PhD)**  
**Asst. Professor**  
**DEPT. OF MCA**

# **EAST WEST INSTITUTE OF TECHNOLOGY, BANGALORE-91**

## **DEPARTMENT OF MCA**

### **VISION**

To provide a quality Education with state-of-art computational facility and to develop quality professionals in Computer Applications as per societal and industrial needs.

### **MISSION**

- Facilitate effective learning environment through quality education, state-of-the-art facilities and orientation towards entrepreneurial skills.
- To promote graduates with opportunities needed to explore, create and face challenges of IT industries.
- To work closely with industry leaders and to help through in undertaking training program needed in their industrial sector.

### **QUALITY POLICY**

- To inculcate the values of DISCIPLINE, DEDECATION and DUTY and inspire the young students to imbibe them scrupulously
- To provide an integrated learning environment to enable students to grow towards their full potential and meet the high expectation of the industry and Society.

# INDEX

| Sl.No | Content Page               |
|-------|----------------------------|
| 1     | List of Lab Programs       |
| 2     | Do's and Don'ts in the Lab |
| 3     | Solutions to Lab Programs  |

## JAVA LABORATORY

|                                |                   |            |    |
|--------------------------------|-------------------|------------|----|
| Course Code                    | PMCAL27           | CIE Marks  | 50 |
| Teaching Hours/Week (L:P: SDA) | 2(Hours per week) | SEE Marks  | 50 |
| Credits                        | 01                | Exam Hours | 03 |

### Course objectives:

- Design & Develop the fundamentals of Object-oriented programming in Java, including defining classes, invoking methods, using class libraries.
- Design & Develop exception handling and multithreading concepts.
- Develop efficient Java applets and applications using OOP concepts

| SL.NO | Experiments                                                                                                                                                                                                                                                                                                                                                                  |
|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1     | Write a Java program to print the following triangle of numbers<br><pre> 1 1 2 1 2 3 1 2 3 4 1 2 3 4 5 </pre>                                                                                                                                                                                                                                                                |
| 2     | Write a Java program to list the factorial of the numbers 1 to 10. To calculate the factorial value, use while loop.(Hint Factof 4 = 4*3*2*1)                                                                                                                                                                                                                                |
| 3     | Write a Java program <ul style="list-style-type: none"> <li>• To find the area and circumference of the circle by accepting the radius from the user.</li> <li>• To accept a number and find whether the number is Prime or not</li> </ul>                                                                                                                                   |
| 4     | Write a Java program to demonstrate a division by zero exception                                                                                                                                                                                                                                                                                                             |
| 5     | Write a Java program to implement Inner class and demonstrate its Access protection.                                                                                                                                                                                                                                                                                         |
| 6     | Write a Java program to demonstrate Constructor Overloading and Method Overloading.                                                                                                                                                                                                                                                                                          |
| 7     | Write a JAVA program to demonstrate Inheritance.                                                                                                                                                                                                                                                                                                                             |
| 8     | Simple Program on Java for the implementation of Multiple inheritance using interfaces to calculate the area of a rectangle and triangle.                                                                                                                                                                                                                                    |
| 9     | Write a program in Java for String handling which performs the following: <ol style="list-style-type: none"> <li>i) Checks the capacity of String Buffer objects.</li> <li>ii) Reverses the contents of string given on console and converts the resultant string in upper case.</li> <li>iii) Reads a string from console and appends it to the resultant string</li> </ol> |

# **LAB PROGRAMS**

# **Do's and Don'ts in the Laboratory**

## **DO .....**

- Come prepared to the Lab.
- Submit your Records to the Lecturer
- Follow the Lab exercise cycles as instructed by the Department.
- Violating the same will result in deduction of marks.
- Put the chairs back to its position before you leave.
- Backlog exercises should be executed after completing regular exercises.

## **DON'T .....**

- Move around in the lab during the lab session.
- Tamper System Files or Try to access the Server.
- Write Data Sheets or Records in the Lab.
- Change the system assigned to you without the notice of the Lab Staff.
- Teaching your friends.

# **SOLUTIONS**

**1. Write a Java program to print the following triangle of numbers**

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

**Source code:**

```
public class Triangle {

    public static void main(String[] args) {

        // Outer loop controls the number of rows

        for (int i = 1; i <= 5; i++) {

            // Inner loop prints numbers from 1 to i

            for (int j = 1; j <= i; j++) {

                System.out.print(j + " ");

            }

            System.out.println();

        }

    }

}
```

**Output:**

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

2. Write a Java program to list the factorial of the numbers 1 to 10. To calculate the factorial value, use while loop. (Hint Factof 4 = 4\*3\*2\*1).

**Source code:**

```
public class Fact {  
  
    // Method to calculate factorial using while loop  
    public static long calculateFactorial(int number) {  
        long result = 1;  
        int i = 1;  
  
        while (i <= number) {  
            result *= i;  
            i++;  
        }  
  
        return result;  
    }  
  
    // Main method  
    public static void main(String[] args) {  
        System.out.printf("%-10s %-20s%n", "Number", "Factorial");  
        System.out.println("-----");  
  
        int num = 1;  
        while (num <= 10) {  
            long fact = calculateFactorial(num);  
            System.out.printf("%-10d %-20d%n", num, fact);  
            num++;  
        }  
    }  
}
```

**Output:**

| Number | Factorial |
|--------|-----------|
| -----  |           |
| 1      | 1         |
| 2      | 2         |
| 3      | 6         |
| 4      | 24        |
| 5      | 120       |
| 6      | 720       |
| 7      | 5040      |
| 8      | 40320     |
| 9      | 362880    |
| 10     | 3628800   |

**3. Write a Java program**

- To find the area and circumference of the circle by accepting the radius from the user.
- To accept a number and find whether the number is Prime or not

**Source code:**

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.IOException;

public class CircleAndPrimeSwitch {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

        System.out.println("Choose an option:");
        System.out.println("1. Calculate area and circumference of a circle");
        System.out.println("2. Check if a number is prime");
        System.out.print("Enter your choice (1 or 2): ");
        int choice = Integer.parseInt(br.readLine());

        switch (choice) {
            case 1:
                System.out.print("Enter the radius of the circle: ");
                double radius = Double.parseDouble(br.readLine());

                double area = Math.PI * radius * radius;
                double circumference = 2 * Math.PI * radius;

                System.out.println("Area of the circle: " + area);
                System.out.println("Circumference of the circle: " + circumference);
                break;

            case 2:
```

```
System.out.print("Enter a number to check if it's prime: ");
int number = Integer.parseInt(br.readLine());

boolean isPrime = true;

if (number <= 1) {
    isPrime = false;
} else {
    for (int i = 2; i <= Math.sqrt(number); i++) {
        if (number % i == 0) {
            isPrime = false;
            break;
        }
    }
}

if (isPrime) {
    System.out.println(number + " is a prime number.");
} else {
    System.out.println(number + " is not a prime number.");
}

break;

default:
    System.out.println("Invalid choice. Please enter 1 or 2.");
}
}
}
```

**Output:**

Choose an option:

1. Calculate area and circumference of a circle
2. Check if a number is prime

Enter your choice (1 or 2): 1

Enter the radius of the circle: 4

Area of the circle: 50.26548245743669

Circumference of the circle: 25.132741228718345

**4. Write a Java program to demonstrate a division by zero exception****Source code:**

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.IOException;
public class DivisionByZeroDemo {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        try {
            System.out.print("Enter the numerator: ");
            int numerator = Integer.parseInt(br.readLine());
            System.out.print("Enter the denominator: ");
            int denominator = Integer.parseInt(br.readLine());
            int result = numerator / denominator;
            System.out.println("Result: " + result);
        } catch (ArithmeticException e) {
            System.out.println("Error: Division by zero is not allowed!");
        } catch (NumberFormatException e) {
            System.out.println("Error: Please enter valid integers.");
        }
    }
}
```

**Output:****Case1:**

```
Enter the numerator: 10
Enter the denominator: 0
Error: Division by zero is not allowed!
```

**Case2:**

```
Enter the numerator: 5
Enter the denominator: 2
Result: 2
```

**5. Write a Java program to implement Inner class and demonstrate its Access protection.****Source code:**

```
public class OuterClass {
    private String outerData = "This is private data from OuterClass";
    class InnerClass {
        void display() {
            System.out.println("Accessing outer class data from inner class: " +
outerData);
        }
    }
    public void useInner() {
        InnerClass inner = new InnerClass();
        inner.display(); // Inner class accessing outer class member
    }
    public static void main(String[] args) {
        OuterClass outer = new OuterClass();
        outer.useInner();
        OuterClass.InnerClass inner = outer.new InnerClass();
        inner.display(); // Can still access private outerData because it's within the same
outer class
    }
}
```

**Output:**

```
Accessing outer class data from inner class: This is private data from OuterClass
Accessing outer class data from inner class: This is private data from OuterClass
```

**6. Write a Java program to demonstrate Constructor Overloading and Method Overloading.****Source Code:**

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.IOException;
public class OverloadingDemo {

    int a, b;

    OverloadingDemo() {
        a = 0;
        b = 0;
        System.out.println("Default constructor: a = " + a + ", b = " + b);
    }

    OverloadingDemo(int x) {
        a = x;
        b = 0;
        System.out.println("Constructor with one argument: a = " + a + ", b = " + b);
    }

    OverloadingDemo(int x, int y) {
        a = x;
        b = y;
        System.out.println("Constructor with two arguments: a = " + a + ", b = " + b);
    }

    void show() {
        System.out.println("Method with no parameters");
    }

    void show(int x) {
        System.out.println("Method with one parameter: " + x);
    }
}
```

```
void show(int x, int y) {
    System.out.println("Method with two parameters: " + x + ", " + y);
}

// Main Method
public static void main(String[] args) throws IOException {
    BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
    int choice;

    do {
        System.out.println("\n--- MENU ---");
        System.out.println("1. Constructor Overloading");
        System.out.println("2. Method Overloading");
        System.out.print("Enter your choice: ");
        choice = Integer.parseInt(br.readLine());

        switch (choice) {
            case 1:
                System.out.println("\nConstructor Overloading:");
                OverloadingDemo obj1 = new OverloadingDemo();
                OverloadingDemo obj2 = new OverloadingDemo(10);
                OverloadingDemo obj3 = new OverloadingDemo(10, 20);
                break;

            case 2:
                System.out.println("\nMethod Overloading:");
                OverloadingDemo obj = new OverloadingDemo(); // Default constructor
                obj.show();
                obj.show(5);
                obj.show(3, 7);
        }
    } while (choice != 0);
}
```

```
        break;

        default:
            System.out.println("Invalid choice! Please enter 1 or 2.");
        }
    } while (choice != 0);
}
}
```

**Output:**

--- MENU ---

1. Constructor Overloading

2. Method Overloading

Enter your choice: 1

Constructor Overloading:

Default constructor: a = 0, b = 0

Constructor with one argument: a = 10, b = 0

Constructor with two arguments: a = 10, b = 20

--- MENU ---

1. Constructor Overloading

2. Method Overloading

Enter your choice: 2

Method Overloading:

Default constructor: a = 0, b = 0

Method with no parameters

Method with one parameter: 5

Method with two parameters: 3, 7

**7. Write a JAVA program to demonstrate Inheritance.****Source code:**

```
// Base class (Parent class)
class Animal {
    void eat() {
        System.out.println("Animal is eating.");
    }

    void sleep() {
        System.out.println("Animal is sleeping.");
    }
}

// Single Inheritance
class Dog extends Animal {
    void bark() {
        System.out.println("Dog is barking.");
    }
}

// Multilevel Inheritance
class Puppy extends Dog {
    void play() {
        System.out.println("Puppy is playing.");
    }
}

// Hierarchical Inheritance
class Cat extends Animal {
    void meow() {
        System.out.println("Cat is meowing.");
    }
}
```

```
}  
// Hybrid Inheritance  
// Here, we simulate hybrid inheritance by combining multiple types.  
class Tiger extends Animal {  
    void roar() {  
        System.out.println("Tiger is roaring.");  
    }  
}  
  
// Main class demonstrating the inheritance types  
public class InheritanceDemo {  
    public static void main(String[] args) {  
  
        // Demonstrating Single Inheritance  
        System.out.println("Single Inheritance:");  
        Dog dog = new Dog();  
        dog.eat(); // Inherited from Animal class  
        dog.bark(); // Defined in Dog class  
        System.out.println();  
  
        // Demonstrating Multilevel Inheritance  
        System.out.println("Multilevel Inheritance:");  
        Puppy puppy = new Puppy();  
        puppy.eat(); // Inherited from Animal class  
        puppy.bark(); // Inherited from Dog class  
        puppy.play(); // Defined in Puppy class  
        System.out.println();  
  
        // Demonstrating Hierarchical Inheritance  
        System.out.println("Hierarchical Inheritance:");  
        Cat cat = new Cat();  
        cat.eat(); // Inherited from Animal class  
        cat.meow(); // Defined in Cat class
```

```
        System.out.println();  
        // Demonstrating Hybrid Inheritance  
        System.out.println("Hybrid Inheritance:");  
        Tiger tiger = new Tiger();  
        tiger.eat(); // Inherited from Animal class  
        tiger.roar(); // Defined in Tiger class  
    }  
}
```

**Output:**

```
>> javac InheritanceDemo.java
```

```
>> java InheritanceDemo
```

Single Inheritance:

Animal is eating.

Dog is barking.

Multilevel Inheritance:

Animal is eating.

Dog is barking.

Puppy is playing.

Hierarchical Inheritance:

Animal is eating.

Cat is meowing.

Hybrid Inheritance:

Animal is eating.

Tiger is roaring.

**8. Simple Program on Java for the implementation of Multiple inheritance using interfaces to calculate the area of a Rectangle and triangle.****Source Code:**

```
// First interface for calculating the area of a rectangle
interface Rectangle {
    double calculateArea(double length, double width); // For rectangle area
}

// Second interface for calculating the area of a triangle
interface Triangle {
    double calculateArea(double base, double height); // For triangle area
}

// Class that implements both interfaces
public class AreaCalculator implements Rectangle, Triangle {

    // Implementing the method from Rectangle interface
    @Override
    public double calculateArea(double length, double width) {
        return length * width; // Area of rectangle = length * width
    }

    // Implementing the method from Triangle interface

    public double calculateArea(double base, float height) {
        return 0.5 * base * height; // Area of triangle = 0.5 * base * height
    }

    public static void main(String[] args) {
        // Creating an object of AreaCalculator
        AreaCalculator calculator = new AreaCalculator();
    }
}
```

```
// Calculating and displaying the area of rectangle
double rectangleArea = calculator.calculateArea(5, 10); // length = 5, width = 10
System.out.println("Area of Rectangle: " + rectangleArea);

// Calculating and displaying the area of triangle
double triangleArea = calculator.calculateArea(6, 8); // base = 6, height = 8
System.out.println("Area of Triangle: " + triangleArea);
}
}
```

**Output:**

Area of Rectangle: 25.0

Area of Triangle: 24.0

**9. Write a program in Java for String handling which performs the following:**

- a. Check the capacity of String Buffer objects.**
- b. Reverse the contents of string given on console and converts the resultant string in upper case.**
- c. Reads a string from console and appends it to the resultant string**

**Source code:**

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.IOException;

public class StringHandling {

    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

        // Initial StringBuffer
        StringBuffer sb = new StringBuffer();

        while (true) {
            // Menu for user input
            System.out.println("\n--- Menu ---");
            System.out.println("1. Check StringBuffer Capacity");
            System.out.println("2. Reverse and Convert to Uppercase");
            System.out.println("3. Append String to Result");
            System.out.print("Enter your choice: ");

            int choice = Integer.parseInt(br.readLine());

            switch (choice) {
                case 1:
                    // Check capacity of StringBuffer
```

```
        System.out.println("Current capacity of StringBuffer: " + sb.capacity());
        break;
    case 2:
        System.out.print("Enter a string to reverse: ");
        String input = br.readLine();
        sb = new StringBuffer(input); // Reset the StringBuffer with input string
        String reversed = sb.reverse().toString(); // Reverse the string
        String upperCaseResult = reversed.toUpperCase(); // Convert to uppercase
        System.out.println("Reversed and Uppercase Result: " + upperCaseResult);
        break;

    case 3:
        System.out.print("Enter a string to append: ");
        String appendStr = br.readLine();
        sb.append(appendStr);
        System.out.println("Resultant String after append: " + sb.toString());
        break;

    default:
        System.out.println("Invalid choice! Please enter a valid option.");
        break;
    }
}
}
```

**Output:**

--- Menu ---

```
1. Check StringBuffer Capacity
2. Reverse and Convert to Uppercase
3. Append String to Result
Enter your choice: 1
Current capacity of StringBuffer: 16
```

--- Menu ---

1. Check StringBuffer Capacity
2. Reverse and Convert to Uppercase
3. Append String to Result

Enter your choice: 2

Enter a string to reverse: avaJ

Reversed and Uppercase Result: JAVA

--- Menu ---

1. Check StringBuffer Capacity
2. Reverse and Convert to Uppercase
3. Append String to Result

Enter your choice: 3

Enter a string to append: Programming

Resultant String after append: Java Programming